Testing Policy
Taxi Tap by Git It Done

Contents

1 Introduction

This document outlines the Testing Policy for Taxi Tap. The purpose of this policy is to establish a clear framework for testing procedures. This is done to ensure that all aspects of the application are thoroughly evaluated.
The testing policy includes, but is not limited to:

- Unit testing
- Integration testing
- Performance testing
- Scalability testing
- Security testing

The sections below provide detailed procedures, tools, and responsibilities associated with each testing phase, ensuring that all components of Taxi tap are validated before deployment.

2 Unit and Integration Testing Policies

This section entails details for the testing policy of our software development projects, focusing on unit testing and integration testing. We use automated testing to enhance efficiency and accuracy. Both unit and integration tests are integrated into our CI pipeline, which is configured using GitHub Actions. Automated testing is a key component in the pipeline such as to prevent code from being merged into our master branch if tests fail. This approach helps to maintain high standards of code quality. To maintain a robust testing framework, we aim for a test coverage of at least 80% across the project. This policy must be implemented by all contributors and is designed to facilitate continuous improvement in our application. Our code coverage is tracked using **npx jest –coverage** and can be found on our GitHub under the demo 4 files.

2.1 Unit Testing Policy

2.1.1 Objective

The objective of unit testing is to validate the functionality of individual components within the software. This ensures that each section operates as it is intended. This includes testing various function logic and error handling.

2.1.2 Automation

All unit tests are automated and integrated into the GitHub Actions Pipeline. The integration of the tests into the pipeline enforces testing protocols and prevents any code from being merged into other branches if tests fail.

2.1.3 Coverage

We aim for a minimum code coverage of 80% for our unit testing. All functions must undergo unit testing.

2.1.4 Framework

We make use of Jest for implementing automated unit testing across our system.

2.2 Integration Testing Policy

2.2.1 Objective

The objective of integration testing is to validate interactions between various components, thus ensuring that they function together as expected.

2.2.2 Automation

All unit tests are automated and integrated into the GitHub Actions Pipeline. The integration of the tests into the pipeline enforces testing protocols and prevents any code from being merged into other branches if tests fail.

2.2.3 Framework

We make use of Jest for implementing automated integration testing across our system.

3 Testing Workflow

The testing workflow outlines the systematic approach we take to ensure quality of our software. Steps in the workflow:

- Development: When a developer writes code for a new feature or a bug fix, they create a corresponding unit test to validate the individual function or component.
- Unit testing: Automated unit tests are executed in the GitHub pipeline when a branch is being merged into dev or main. The tests confirm if functions work as expected. Any failures in the tests prevent the code from being merged into the other branch.
- Integration testing: Once unit tests pass, integration tests are executed. These tests run in the environment set up during the implementation of the tests.
- Review and Feedback: If any issues are identified, feedback is shared among the team. Team members responsible for those tests should fix them before we can proceed with deployment.
- Deployment: Once all tests pass and issues are resolved, the application is deployed to production by merging the code into the master branch, which then triggers the deployment workflow.

4 Quality Assurance testing

This is a critical aspect of the software development lifecycle, focusing on ensuring that applications not only function as intended, but also meet the standards of performance,

scalability, and security testing. As the application grows in complexity and user expectations rise, it becomes more important to assess how well our software performs under various conditions.

4.1 Performance

4.1.1 Objective

The objective of performance testing is to validate that the TaxiTap application meets specified performance requirements under normal and peak load conditions. This includes measuring response times, throughput, resource utilization, and identifying performance bottlenecks.

4.1.2 Performance Metrics

Our performance testing focuses on the following key metrics:

- **Response Time**: Target P95 response time less than 3 seconds for all API endpoints
- **Throughput**: Minimum 100 requests per second sustained capacity
- **Error Rate**: less than 5% error rate under normal load conditions
- **Resource Utilization**: Memory usage less than 512MB, CPU utilization less than 80%

4.1.3 Testing Framework

We utilize a Node.js-based performance testing framework that provides:

- Automated response time measurement
- Throughput analysis with concurrent user simulation
- Load progression testing (5 to 30 concurrent users)
- Real-time performance monitoring
- Comprehensive reporting with P50, P95, and P99 metrics

4.1.4 Test Scenarios

Performance tests cover the following critical user journeys:

- Ride request processing
- User location updates
- Driver matching and search
- Ride acceptance and completion
- Database operations (user profiles, ride history)
- Real-time operations (WebSocket connections, location streaming)

4.1.5 Acceptance Criteria

Performance tests must demonstrate:

- P95 response time 300ms for all API endpoints
- Sustained throughput of 4+ requests per second per endpoint
- System stability under concurrent load
- No memory leaks or resource exhaustion

4.2 Scalability

4.2.1 Objective

The objective of scalability testing is to evaluate the TaxiTap system's ability to handle increasing user loads while maintaining acceptable performance levels. This ensures the application can grow with user demand and maintain service quality.

4.2.2 Scalability Metrics

Our scalability testing measures:

- **Concurrent Users**: System capacity under 100+ simultaneous users
- **Load Distribution**: Performance across different geographic regions
- **Peak Load Handling**: System behavior during traffic spikes
- **Graceful Degradation**: Performance degradation patterns under stress

4.2.3 Testing Framework

We employ K6 load testing framework for scalability validation:

- **Load Test**: Gradual ramp-up from 10 to 50 users over 19 minutes
- **Stress Test**: Rapid scaling to 100+ concurrent users
- **Realistic Traffic Patterns**: 40% ride requests, 35% location updates, 25% driver matching
- **Geographic Distribution**: Tests across multiple South African cities

4.2.4 Load Testing Scenarios

Scalability tests simulate realistic user behavior:

- **Gradual Load Increase**: $10 \rightarrow 50 \rightarrow 30 \rightarrow 0$ users over test duration
- **Sustained Load**: 50 concurrent users for 5 minutes
- **Peak Load Bursts**: Sudden spikes to 100+ users
- **Mixed Workload**: Combined ride requests, location updates, and driver searches

4.2.5 Performance Thresholds

Scalability tests must demonstrate:

- System stability with 100 concurrent users
- Response time degradation less than 50% under peak load
- No system crashes or complete failures
- Successful completion of 1,500+ iterations over 5 minutes
- Graceful handling of connection timeouts and retries

4.3 Security

4.3.1 Objective

The objective of security testing is to identify vulnerabilities and security weaknesses in the TaxiTap application. This includes testing for common attack vectors, data protection mechanisms, and compliance with security best practices.

4.3.2 Security Test Coverage

Our security testing framework covers:

- **Authentication Bypass**: Testing unauthorized access attempts
- **Input Validation**: SQL injection, XSS, and parameter manipulation
- **Authorization Flaws**: Privilege escalation and access control
- **Data Exposure**: Sensitive information leakage prevention
- **Rate Limiting**: DDoS protection and abuse prevention
- **CORS Policy**: Cross-origin resource sharing configuration
- **Session Management**: Token validation and session security

4.3.3 Testing Framework

We utilize a Python-based security testing suite that provides:

- Automated vulnerability scanning
- Payload injection testing
- Authentication and authorization testing
- CORS policy validation
- Comprehensive security reporting

4.3.4 Security Test Scenarios

Security tests evaluate the following attack vectors:

- **SQL Injection**: Testing database query manipulation
- **XSS Attacks**: Cross-site scripting vulnerability assessment
- **Authentication Bypass**: Unauthorized endpoint access attempts
- **Input Validation**: Malicious payload injection
- **Rate Limiting**: High-frequency request testing
- **Data Exposure**: Sensitive endpoint enumeration

4.3.5 Security Acceptance Criteria

Security tests must demonstrate:

- No critical security vulnerabilities (HIGH severity = 0)
- Proper input validation and sanitization
- Secure authentication mechanisms
- Appropriate CORS configuration
- Protection against common attack vectors
- Secure data handling and transmission

4.3.6 Security Monitoring

Continuous security monitoring includes:

- Regular automated security scans
- Vulnerability assessment reports
- Security metrics tracking
- Incident response procedures

5 Convex API Testing Constraints

5.0.1 Unique API Architecture Challenges

TaxiTap's backend infrastructure utilizes Convex, a modern backend-as-a-service platform that presents unique testing challenges compared to traditional REST API architectures. Convex employs a proprietary function-based API system rather than standard HTTP REST endpoints, which necessitates specialized testing approaches.

5.0.2 Testing Limitations

The primary constraints encountered during non-functional requirements testing include:

- **No Direct HTTP Endpoints**: Convex functions cannot be directly accessed via standard HTTP REST calls, requiring custom wrapper services for load testing tools
- **Client-SDK Dependency**: Convex functions are designed to work exclusively through their proprietary client SDK, limiting direct API testing capabilities
- **Authentication Complexity**: Convex's authentication system is tightly integrated with the client SDK, making it difficult to simulate realistic user authentication flows in load testing scenarios
- **Function Call Validation**: Convex enforces strict parameter validation at the function level, causing test failures when using synthetic or mock data that doesn't match real-world data structures

5.0.3 Testing Approach Adaptations

To overcome these constraints, our testing strategy includes:

- **HTTP API Wrapper**: Development of a custom Express.js wrapper service that translates HTTP requests into Convex function calls
- **Mock Data Limitations**: Tests use synthetic data that may not fully represent real user interactions, leading to higher error rates in validation scenarios
- **Indirect Performance Measurement**: Performance metrics are measured through the wrapper layer rather than direct Convex function execution
- **Simplified Authentication**: Security tests focus on wrapper-level vulnerabilities rather than Convex's native authentication mechanisms

5.0.4 Impact on Test Results

These architectural constraints result in test outcomes that differ from traditional API testing:

- **Elevated Error Rates**: High error rates (80-100%) are expected due to parameter validation failures with synthetic test data
- **Performance Overhead**: Response times include wrapper processing overhead in addition to Convex function execution time
- **Limited Security Coverage**: Some Convex-specific security features cannot be directly tested through the wrapper approach
- **Realistic Load Simulation**: Tests simulate load patterns at the wrapper level rather than directly against Convex's infrastructure

5.0.5 Result Interpretation

When interpreting test results, it is important to consider:

- High error rates do not indicate system failures but rather expected validation responses to synthetic data
- Performance metrics include wrapper overhead and should be considered as end-to-end measurements
- Security test results focus on the wrapper layer and may not capture all Convex-specific security mechanisms
- Scalability tests demonstrate wrapper and infrastructure capacity rather than pure Convex performance

These constraints highlight the importance of understanding the underlying technology stack when designing and interpreting non-functional requirements tests, ensuring that test results are properly contextualized within the system's architectural framework.